DeepLOB Reproduction on FI-2010 and Transfer to Optiver Limit Order Book Data

CS 289 Final Project Team

Spring 2026

Abstract

We study whether a deep neural architecture originally proposed for limit order book (LOB) forecasting, DeepLOB, can be reproduced on the FI-2010 benchmark and then transferred to a materially different market dataset from Optiver. Our project has two linked goals. First, we reproduce the core CNN–Inception–LSTM pipeline on FI-2010 and verify that it achieves strong mid-price direction classification performance across multiple event horizons. Second, we adapt the architecture to Optiver's two-level book representation, introduce causal normalization and transfer-learning protocols, and evaluate zero-shot generalization, per-stock fine-tuning, and same-stock out-of-sample training. On FI-2010, the reproduced model reaches its strongest performance at longer horizons, with accuracy rising from 0.766 at 10 events to roughly 0.795 at 100 events and Matthews correlation coefficient (MCC) rising from 0.426 to 0.692. We further analyze engineered microstructure factors, show that price/volume derivative features are consistently predictive, and confirm that DeepLOB outperforms shallow manual-factor baselines. On Optiver, however, transfer is much harder: a universal model produces limited zero-shot generalization, and fine-tuning improves agreement metrics more than raw accuracy, revealing a substantial domain-shift problem rather than a simple optimization issue. The resulting report presents both positive and negative findings: DeepLOB is reproducible on FI-2010, but naive transfer to Optiver is unstable and motivates dataset-specific modeling changes.

1 Problem Definition

This project asks two connected technical questions:

1. Can we faithfully reproduce the main empirical behavior of DeepLOB on the FI-2010 benchmark using a modern PyTorch implementation?

2. If so, how well does the same modeling idea transfer to a different LOB environment, namely Optiver's two-level auction book data, after architectural adaptation and fine-tuning?

These questions are worth studying for two reasons. First, LOB prediction is a canonical sequential learning problem with strong local structure, temporal dependence, and severe class imbalance, making it a natural testbed for course ideas in representation learning and generalization. Second, successful performance on a benchmark does not guarantee transfer to a new market or data schema. Understanding where transfer succeeds and fails is practically important and scientifically useful.

Our final deliverable is therefore not just a reproduction. It is a controlled study of how far the DeepLOB design carries beyond FI-2010, and what breaks when the data distribution, label process, and book structure change.

2 Connection to Course Content

This project is grounded in several topics from CS 289:

- **Supervised learning and empirical risk minimization.** Both FI-2010 and Optiver tasks are formulated as multi-class classification of future mid-price direction from recent LOB states.
- **Deep sequence models.** The model combines convolutional feature extraction with recurrent temporal aggregation, illustrating how inductive bias matters for structured time series.
- **Optimization and regularization.** We use Adam, weight decay, dropout, early stopping, gradient clipping, and label smoothing, and we explicitly study how optimizer choices affect stability.
- **Class imbalance and evaluation.** Accuracy alone is misleading for LOB data, so we track macro-F1, Cohen's κ, and MCC. This directly reflects course discussions about metric choice under skewed labels.
- **Transfer learning and distribution shift.** The Optiver portion tests zero-shot transfer, head-only versus deeper fine-tuning, and stock-level heterogeneity.
- **Baselines and ablations.** We compare DeepLOB against logistic regression and an MLP on engineered features, and we compare universal, fine-tuned, and stock-specific Optiver models under matched evaluation.

3 Prior Work

DeepLOB, introduced by Zhang, Zohren, and Roberts [3], is a well-known architecture for high-frequency LOB forecasting. Its core idea is to combine local convolutional pattern extraction, an Inception-style multi-scale module, and an LSTM that aggregates short-term temporal structure before classification. The benchmark most closely associated with this line of work is FI-2010, curated by Ntakaris et al. [2], which provides a standardized setup for mid-price movement prediction across several event horizons.

Earlier work on LOB prediction often relied on handcrafted microstructure features and simpler classifiers. Kercheval and Zhang [1], for example, studied machine learning methods on LOB-derived features, highlighting that engineered factors such as imbalance and order-flow statistics can contain predictive information. This matters for our project because part of our contribution is to compare DeepLOB with manual-factor baselines instead of treating the deep model as a black box.

More broadly, financial time-series forecasting is known to be sensitive to nonstationarity, label definition, and market-specific effects. Methods that work on one dataset can degrade sharply under even moderate shifts in trading regime, tick structure, or normalization scheme. That concern motivates our Optiver study. Rather than proposing a novel architecture from scratch, we investigate whether a proven LOB model can generalize to a new setting with only modest adaptation.

4 Data and Experimental Setup

4.1 FI-2010

FI-2010 contains normalized limit order book data with 40 input features corresponding to 10 bid/ask levels and associated volumes. Following the standard setup, the task is three-class classi-

fication of future mid-price movement at five event horizons: 10, 20, 30, 50, and 100 events ahead. In the codebase, these appear as horizon indices $k \in \{0, \ldots, 4\}$.

The training pipeline uses a lookback window of 100 events. The repository splits the provided training set into train and validation partitions and evaluates on the concatenated benchmark test files. Although the final report focus is the retained $k = 3, 4, 5$ slice (30, 50, and 100 events), the active results directory contains the full five-horizon evaluation, which we summarize for completeness.

4.2 Optiver

The Optiver dataset is substantially different. Each event contains only two order book levels, yielding 8 raw LOB features after reordering:

$$\{\texttt{ask_price1}, \texttt{ask_size1}, \texttt{bid_price1}, \texttt{bid_size1}, \texttt{ask_price2}, \texttt{ask_size2}, \texttt{bid_price2}, \texttt{bid_size2}\}.$$

Labels are built from future mid-price returns, and preprocessing applies *causal* normalization so that each event is standardized only from past information. This is an important design choice: it avoids lookahead leakage that would otherwise be easy to introduce in event-wise normalization.

The retained Optiver report slice focuses on the 5-event horizon. The saved artifacts evaluate five held-out transfer stocks, and the corresponding result files in the repository are for stock IDs 90, 99, 108, 116, and 126.

4.3 Evaluation Metrics

Across experiments we report accuracy, Cohen's κ, MCC, and class-wise or weighted F1 scores. For Optiver we also compare regime-style profit proxies included by the project scripts, but we treat those as secondary diagnostic metrics rather than direct evidence of a deployable trading strategy.

5 Approach

5.1 DeepLOB Reproduction on FI-2010

Our FI-2010 model follows the DeepLOB design closely. The input tensor has shape $(B, 1, T, 40)$ with $T = 100$. The network contains:

- three convolutional blocks that progressively merge local price-volume structure,
- an Inception-style multi-branch module that captures patterns at multiple receptive fields,
- a one-layer LSTM with hidden size 64,
- a dropout layer and final fully connected classifier over the three movement classes.

The implementation intentionally preserves the architectural logic of the paper while stabilizing optimization. In our environment, a literal paper-style optimizer was unreliable and often collapsed to the majority stationary class. We therefore use a tuned training profile with Adam, horizon-specific regularization, early stopping, optional class weighting, and monitor choices that differ across horizons. Shorter horizons use metrics such as macro-F1 or MCC to avoid stationary-class collapse, while longer horizons monitor validation loss.

5.2 Microstructure Analysis on FI-2010

Beyond reproducing DeepLOB, the project includes a CPU-side analysis pipeline for engineered factors. This pipeline:

- runs Newey–West predictability tests on engineered LOB features,
- checks rolling-window significance stability,
- measures monotonicity and rank information coefficient behavior,
- applies Benjamini–Hochberg false discovery rate control,
- trains shallow baselines on manual factors,
- computes simple strategy-style statistics from model predictions.

This extends the project from "does the network work?" to "what information seems predictive, and how much value does the deep model add over simpler alternatives?"

5.3 DeepLOBLite for Optiver

To transfer the method to Optiver, we adapt the network to the two-level book. The main architectural change is purely at the input interface: the model consumes $(B, 1, T, 8)$ instead of $(B, 1, T, 40)$, and the third convolutional block uses a $(1, 2)$ kernel rather than the original width designed for 10 levels. The LSTM and classifier dimensions are otherwise kept the same. We refer to this adapted model as **DeepLOBLite**.

The Optiver pipeline has three stages:

1. **Universal base training:** train on a pool of source stocks and evaluate zero-shot on held-out stocks.
2. **Transfer learning:** fine-tune the universal model separately for each held-out stock, typically updating the prediction head or a slightly deeper subset of layers.
3. **Specific-stock out-of-sample training:** train a model on an early temporal segment of one stock and evaluate on a later segment of the same stock.

For the retained 5-event Optiver slice, the configuration uses focal loss, dropout 0.30, weight decay, gradient clipping, macro-F1-based monitoring, and a small auxiliary regression head on future log return. The intent is to reduce majority-class collapse while preserving the original classification target.

6 Our Contribution

Our contribution has three distinct parts.

6.1 Contribution 1: A Reproducible DeepLOB Pipeline with Stability Fixes

We reimplemented the end-to-end DeepLOB workflow in a standalone training script and documented a practical issue that is easy to miss in paper reproduction: the nominal architecture is not enough if the optimizer configuration is unstable. In our environment, naive settings led to exploding validation loss and stationary-class collapse. We addressed this with horizon-aware regularization, better monitoring metrics, and safer training defaults. This is a modest but real contribution because it turns a brittle reproduction into a reliable experimental system.

6.2 Contribution 2: Extending the FI-2010 Study Beyond Raw Benchmark Accuracy

Instead of stopping at classification scores, we asked what the dataset itself reveals about microstructure signals. We tested 104 engineered factors, measured statistical significance with Newey–West corrections, screened them with rolling stability and false discovery rate control, and compared the deep model against manual-factor baselines. This produces a more interpretable picture of what information is present in the benchmark and how DeepLOB relates to simpler alternatives.

6.3 Contribution 3: Transferring DeepLOB to a New Dataset and Evaluating Failure Modes

The main original research component is the Optiver study. We adapted DeepLOB to a new LOB representation, built a causal preprocessing pipeline, and evaluated three regimes under the same retained horizon:

- universal zero-shot transfer,
- per-stock fine-tuning,
- stock-specific temporal generalization.

This is exactly the kind of modest but meaningful project extension described in the course rubric: a known method applied to a new dataset, with careful evaluation. Importantly, the result is not uniformly positive. The transfer pipeline exposes significant domain shift, and the negative result itself is a central finding of the project.

7 Results and Discussion

7.1 FI-2010 Classification Performance

table 1 summarizes the reproduced DeepLOB results. Performance improves substantially as the prediction horizon gets longer. While 10-event prediction remains difficult and skewed toward the stationary class, the 50- and 100-event tasks produce much stronger balanced metrics.

Table 1: DeepLOB performance on FI-2010 across benchmark event horizons.

Horizon	Accuracy	Cohen's κ	MCC	Weighted F1
10 events	0.7656	0.3988	0.4255	0.7176
20 events	0.7487	0.4842	0.5022	0.7300
30 events	0.7752	0.6041	0.6076	0.7698
50 events	0.7972	0.6782	0.6792	0.7954
100 events	0.7948	0.6921	0.6923	0.7952

Two patterns stand out. First, MCC and κ increase almost monotonically with horizon, suggesting that longer-horizon labels are less dominated by ambiguous stationary states. Second, the best balanced performance occurs at the 100-event horizon even though raw accuracy slightly peaks at 50 events. This is why including agreement-based metrics matters.

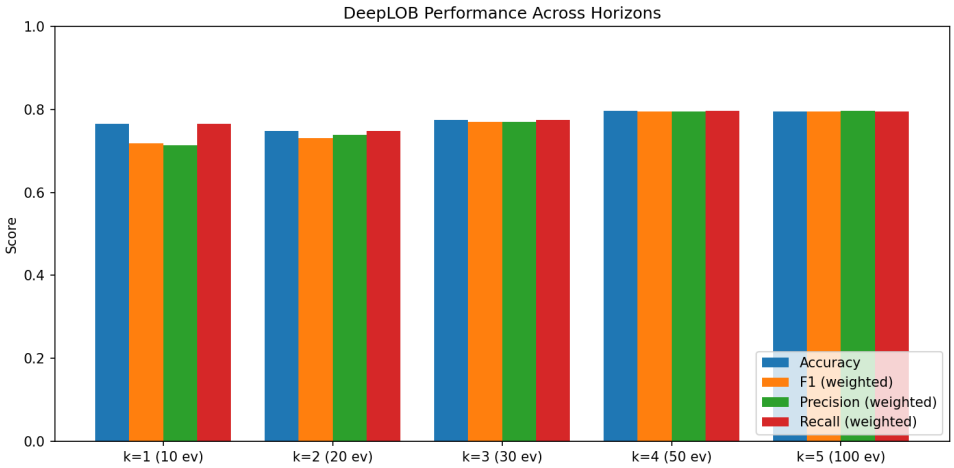
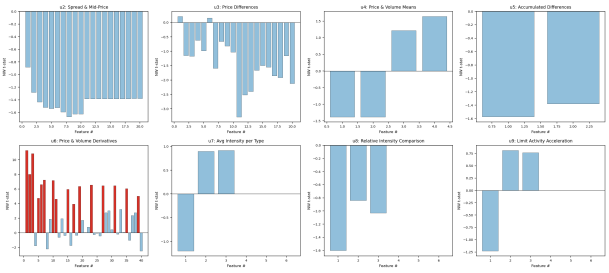


Figure 1: Comparison of FI-2010 performance across horizons from the project analysis pipeline.

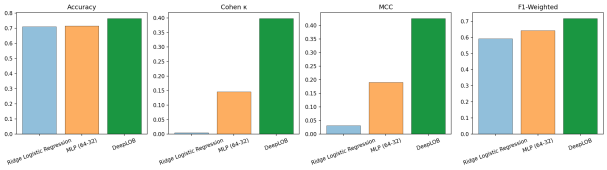
7.2 FI-2010 Baselines and Factor Analysis

The manual-factor baselines underperform the deep model. For the short-horizon slice included in the baseline comparison file, ridge logistic regression reaches accuracy 0.7099 with near-zero κ, and a two-layer MLP reaches accuracy 0.7156 with MCC 0.1908. DeepLOB improves accuracy to 0.7656 and MCC to 0.4255. This gap suggests that the deep architecture is not merely matching what linear or shallow nonlinear models can already extract from a fixed factor set.

The factor analysis provides a complementary view. Out of 104 engineered features tested, 21 survive the qualification pipeline recorded in `qualified_factors.csv`. Most of the strongest signals belong to the "price & volume derivatives" group (set `u6`), and several show very large Newey–West statistics. For example, feature 1 in this group has $t_{\mathrm{NW}} = 11.29$, and feature 3 has $t_{\mathrm{NW}} = 10.84$. Rolling-window analysis also shows that many of these factors remain significant across a large fraction of windows, indicating that their predictive content is not driven by a single isolated regime.



(a) Newey–West feature significance



(b) Baseline model comparison

Figure 2: FI-2010 interpretability and baseline comparison results.

One honest limitation is that the strategy-style pseudo-PnL diagnostics are weak. The saved `trading_strategy_stats.csv` file shows no statistically significant positive trading result, and the longer horizons are actually negative under the simplistic signal-to-PnL mapping. We view this as expected rather than disappointing: classification improvement on a benchmark does not automatically imply profitable trading once action mapping and market frictions are considered.

7.3 Optiver Transfer Results

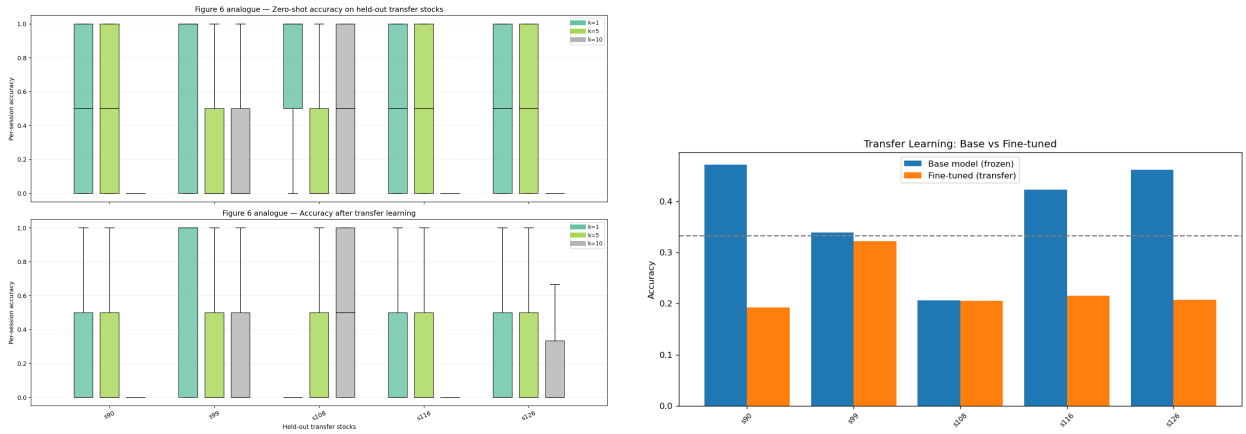
The Optiver results are the most informative part of the project because they reveal where transfer fails. For the retained 5-event slice, the mean zero-shot accuracy of the universal model across the five held-out stocks is 0.3782, with essentially zero average κ (0.0005). This already suggests that the model has limited cross-stock generalization.

Fine-tuning does not improve raw accuracy on average. In fact, the mean accuracy drops from 0.3804 to 0.2284 across the five transfer-study stocks. However, average κ increases from 0.0007 to 0.0146. That combination is unusual but meaningful: fine-tuning appears to move the model away from a trivial dominant-class predictor, yet the resulting predictions still do not align well enough with the true label distribution to improve accuracy. In other words, the problem is not just underfitting; it is label mismatch and distribution shift.

Training a stock-specific model from scratch on an earlier temporal segment performs somewhat better than fine-tuning on average, with mean accuracy 0.3509 and mean κ 0.0216, but it still falls far short of the FI-2010 benchmark behavior.

Table 2: Optiver 5-event horizon comparison across evaluation regimes.

Regime	Mean Accuracy	Mean Cohen's κ	Mean MCC
Universal zero-shot	0.3782	0.0005	0.0047
Per-stock fine-tuned	0.2284	0.0146	–
Specific-stock OOS	0.3509	0.0216	0.0222



(a) Accuracy before and after transfer

(b) Universal vs transfer vs stock-specific

Figure 3: Optiver transfer-learning comparison for the retained 5-event study.

7.4 Why Did Transfer Perform Poorly?

We believe the weak Optiver transfer performance has several causes.

- **Representation mismatch.** FI-2010 uses 10 levels while Optiver exposes only 2. Even after architectural adaptation, the available depth of queue information is dramatically smaller.
- **Label mismatch.** The target is still future direction classification, but the return distribution and thresholding behavior differ by stock and by dataset.

- **Cross-stock heterogeneity.** The held-out Optiver stocks do not appear to share one stable decision boundary. A universal model often collapses to one dominant class, and fine-tuning can over-correct into another dominant class.
- **Temporal nonstationarity.** Same-stock out-of-sample training performs only slightly better, suggesting that within-stock time drift is also substantial.

This negative result is important. It shows that a benchmark-winning LOB architecture is not plug-and-play across datasets, even when the task description sounds similar.

8 Code and Reproducibility

The codebase is organized to separate heavy computation from presentation:

- `scripts/train_deeplob.py` trains DeepLOB on FI-2010.
- `scripts/analyze_fi2010.py` produces feature tests, baselines, and trading-style diagnostics.
- `scripts/prepare_optiver.py` preprocesses Optiver data with causal normalization.
- `scripts/train_optiver.py` runs universal training, transfer learning, and stock-specific evaluation.
- `scripts/analyze_optiver.py` builds notebook-ready summary figures.
- the top-level notebooks load saved artifacts and present the experiments in report form.

This structure supports reproducibility because the expensive stages are scripted, parameterized, and saved to disk, while the notebooks act as a readable front end rather than hiding the main logic.

9 Conclusion

This project reproduced DeepLOB successfully on FI-2010 and showed that the model's strongest benchmark performance occurs at medium-to-long horizons. We also extended the benchmark study with statistically grounded factor analysis and shallow baselines, finding that manual microstructure features contain real signal but still underperform the deep architecture.

Our main original contribution was transferring the method to Optiver. That experiment surfaced a more nuanced conclusion than a simple success story: the architecture transfers imperfectly, stock heterogeneity is strong, and fine-tuning can improve agreement metrics without improving accuracy. The project therefore demonstrates both the strength of DeepLOB as a benchmark model and the limits of benchmark-to-real-dataset generalization.

Project Summary

In one sentence, this project:

> reproduced a canonical deep LOB predictor on FI-2010, analyzed what signals drive performance, and tested how far the same idea transfers to Optiver through causal preprocessing, model adaptation, and transfer-learning experiments.

References

[1] Alec N. Kercheval and Yuan Zhang. Modelling high-frequency limit order book dynamics with support vector machines. *Quantitative Finance*, 15(8):1315–1329, 2015.

[2] Apostolos Ntakaris, Mikko Magris, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data. *Journal of Forecasting*, 37(8):852–866, 2018.

[3] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deeplob: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019.